

Vietnam National University, Ho Chi Minh City
University of Technology
Faculty of Computer Science and Engineering



Microprocessors–Microcontrollers (Lab) (CO3010)

Laboratory Report

LAB_02: TIMER INTERRUPT AND LED SCANNING

Instructor(s): Phan Văn Sỹ, CSE-HCMUT
Class: CC03
Student: Vo Le Thanh Trieu – 2453297

Ho Chi Minh City, October 2026



Contents

1	Timer Interrupt and LED Scanning	3
1.1	Exercise 1: Two seven-segment displays	3
1.1.1	Objective and GPIO configuration	3
1.1.2	Schematic	4
1.1.3	Step-by-step timer calculation	4
1.1.4	Segment-display function	5
1.1.5	Initialization in main	6
1.1.6	Timer interrupt callback	7
1.1.7	Expected simulation behavior	8
1.1.8	Short question: Frequency of the scanning process	8
1.2	Exercise 2: Four digits and the center dot	9
1.2.1	Objective and GPIO configuration	9
1.2.2	Schematic	9
1.2.3	Step-by-step timing calculation	9
1.2.4	Segment-display function	10
1.2.5	Initialization in main	12
1.2.6	Timer interrupt callback	12
1.2.7	Expected simulation behavior	14
1.2.8	Short question: Frequency of the scanning process	14
1.3	Exercise 3: Buffered digit selection	15
1.3.1	Objective and circuit	15
1.3.2	Buffer declarations and function prototypes	15
1.3.3	Digit encoding for values 0–9	16
1.3.4	Report 1: update7SEG function	17
1.3.5	Initialization in main	18
1.3.6	Report 2: Timer interrupt callback	18
1.3.7	Expected behavior and buffer testing	19
1.4	Exercise 4: One complete scan per second	20
1.4.1	Objective and circuit	20
1.4.2	Required scan timing	20
1.4.3	Initialization in main	21
1.4.4	Report 1: Timer interrupt callback	21
1.4.5	Expected simulation behavior	22
1.5	Exercise 5: Digital clock HH:MM	23
1.5.1	Objective and inherited configuration	23
1.5.2	Time variables and display buffer	23
1.5.3	Report 1: updateClockBuffer function	23
1.5.4	Initialization	23
1.5.5	Clock update in the main loop	24
1.5.6	Expected behavior	24
1.6	Exercise 6: Software timer for the red LED	25
1.6.1	Objective and inherited configuration	25
1.6.2	New variables and software-timer functions	25
1.6.3	Updated timer interrupt callback	26
1.6.4	Initialization	26
1.6.5	Updated main loop	27
1.6.6	Answers to the three report questions	28
1.6.7	Expected behavior	28
1.7	Exercise 7: Clock and DOT handled in main	28
1.7.1	Objective and inherited configuration	28
1.7.2	Software-timer extension	28
1.7.3	Updated timer interrupt callback	29



1.7.4	Initialization	30
1.7.5	Report 1: Main-loop source code	30
1.7.6	Expected behavior and timer usage	31
1.8	Exercise 8: Software-timer-only TIM2 callback	31
1.8.1	Objective and inherited configuration	31
1.8.2	Additional software timer	32
1.8.3	Updated timer interrupt callback	33
1.8.4	Initialization	33
1.8.5	Report 1: Main-loop source code	33
1.8.6	Expected behavior	34
1.9	Exercise 9: Display the letter A on an 8×8 matrix	35
1.9.1	Objective and inherited configuration	35
1.9.2	Updated matrix GPIO configuration	35
1.9.3	Report 1: Schematic	36
1.9.4	Matrix variables and character pattern	36
1.9.5	Report 2: updateLEDMatrix function	36
1.9.6	Complete application initialization	38
1.9.7	Complete main loop	38
1.9.8	Expected behavior	40
1.10	Exercise 10: Scroll the matrix character to the left	40
1.10.1	Objective and inherited configuration	40
1.10.2	Additional animation variables	40
1.10.3	Report 1: Left-scrolling function	41
1.10.4	Complete application initialization	41
1.10.5	Complete main loop	42
1.10.6	Expected behavior	43
References		44



1 Timer Interrupt and LED Scanning

This report follows Exercises 1–10 of *Lab 02: Timer Interrupt and LED Scanning*.

STM32CubeMX configures the STM32F103C6; application code is edited and built in Visual Studio Code with CMake. Proteus provides the circuit simulation. The code listings document successive versions of the same application: later exercises extend the earlier ones. Screenshots of the student's own Proteus simulations can be inserted into the named figure files.

1.1 Exercise 1: Two seven-segment displays

1.1.1 Objective and GPIO configuration

Exercise 1 uses timer interrupts to control two common-anode seven-segment displays. The first display shows the number 1, and the second shows the number 2. Only one display is enabled at a time, and the selected display changes every 500 ms.

The segment cathodes of both displays share PB0–PB6. Separate PNP transistors control their common anodes through PA6 and PA7. This arrangement allows the two displays to share the same segment signals.

Table 1: GPIO configuration for Exercise 1

GPIO pins	Signals	Function
PB0–PB6	SEG_A–SEG_G	Shared segment outputs, active low
PA6	EN0	First display enable, active low
PA7	EN1	Second display enable, active low

PB0, PB1, PB2, PB3, PB4, PB5, and PB6 correspond to segments a, b, c, d, e, f, and g, respectively. These pins are configured as push-pull outputs with no internal pull-up or pull-down resistors.

For the selected common-anode display, writing `GPIO_PIN_RESET` to a segment output turns that segment on. Writing `GPIO_PIN_SET` turns it off. The PNP enable signals are also active low: PA6 or PA7 must be low to enable its corresponding display.

1.1.2 Schematic

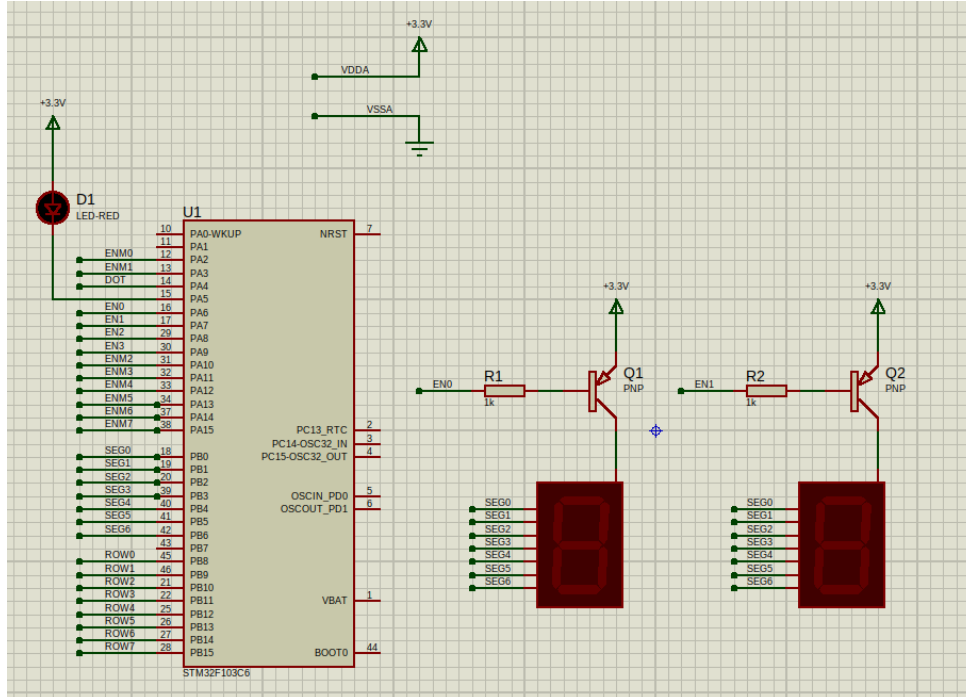


Figure 1: Proteus schematic for Exercise 1: two common-anode digits and PNP enables.

1.1.3 Step-by-step timer calculation

The timer configuration follows the introductory timer setup in the lab manual. TIM2 generates an interrupt every 10 ms, and the application counts these interrupts to obtain the required 500-ms display-selection interval.

1.1.3.1 Step 1: Determine the TIM2 input clock. The source code selects the internal HSI oscillator, disables the PLL, and sets the AHB and APB1 clock dividers to 1. Therefore,

$$f_{\text{SYSCLK}} = f_{\text{HCLK}} = f_{\text{PCLK1}} = 8\,000\,000 \text{ Hz.}$$

Because the APB1 divider is 1, the TIM2 input clock equals the APB1 peripheral clock:

$$f_{\text{TIM2}} = 8\,000\,000 \text{ Hz} = 8 \text{ MHz.}$$

1.1.3.2 Step 2: Calculate the counter clock after the prescaler. The source code sets

$$PSC = 7999.$$

The prescaler divides the timer input clock by $PSC + 1$, so the counter frequency is

$$f_{\text{CNT}} = \frac{f_{\text{TIM2}}}{PSC + 1} = \frac{8\,000\,000}{7999 + 1} = \frac{8\,000\,000}{8000} = 1000 \text{ Hz.}$$

Thus, the counter advances once every

$$T_{\text{CNT}} = \frac{1}{f_{\text{CNT}}} = \frac{1}{1000} \text{ s} = 1 \text{ ms.}$$

1.1.3.3 Step 3: Calculate the timer update interval. TIM2 uses up-counting mode, and the source code sets

$$ARR = 9.$$

The counter counts from 0 through 9. An update occurs when it overflows and returns to 0, giving

$$N_{\text{counts}} = ARR + 1 = 10$$

counter ticks per update.

Therefore,

$$T_{\text{interrupt}} = (ARR + 1)T_{\text{CNT}} = 10 \times 1 \text{ ms} = 10 \text{ ms}.$$

Equivalently,

$$T_{\text{interrupt}} = \frac{(PSC + 1)(ARR + 1)}{f_{\text{TIM2}}} = \frac{8000 \times 10}{8\,000\,000} = 0.01 \text{ s}.$$

1.1.3.4 Step 4: Calculate the interrupt frequency. The timer interrupt frequency is

$$f_{\text{interrupt}} = \frac{1}{T_{\text{interrupt}}} = \frac{1}{0.01} = 100 \text{ Hz}.$$

With TIM2 interrupts enabled, the callback is therefore invoked 100 times per second.

1.1.3.5 Step 5: Calculate the number of interrupts per display. Each display must remain selected for

$$T_S = 500 \text{ ms}.$$

The required number of timer interrupts is

$$N_{\text{interrupts}} = \frac{T_S}{T_{\text{interrupt}}} = \frac{500 \text{ ms}}{10 \text{ ms}} = 50.$$

This explains the condition `ticks >= 50` in the callback. The hardware timer still interrupts every 10 ms, while the display changes every

$$50 \times 10 \text{ ms} = 500 \text{ ms}.$$

1.1.4 Segment-display function

The actual `display7SEG()` implementation in Exercise 1 handles only the numbers 1 and 2.

Before writing a new digit, the function sets PB0–PB6 high to turn off all segments. It then drives the required segment outputs low:

- Number 1 uses segments b and c: PB1 and PB2.
- Number 2 uses segments a, b, d, e, and g: PB0, PB1, PB3, PB4, and PB6.

The prototype is placed in `USER CODE BEGIN PFP`, and the function definition is placed in `USER CODE BEGIN 0`.

Program 1.1: Exercise 1 display7SEG function

```
1  /* USER CODE BEGIN PFP */
2  void display7SEG(int num);
3  /* USER CODE END PFP */
4
```

```

5  /* USER CODE BEGIN 0 */
6  void display7SEG(int num){
7      /* Turn off all seven segments first */
8      HAL_GPIO_WritePin(
9          GPIOB,
10         GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_2 | GPIO_PIN_3 |
11         GPIO_PIN_4 | GPIO_PIN_5 | GPIO_PIN_6,
12         GPIO_PIN_SET
13     );
14
15     if (num == 1)
16     {
17         /* Number 1 uses segments b and c */
18         HAL_GPIO_WritePin(
19             GPIOB,
20             GPIO_PIN_1 | GPIO_PIN_2,
21             GPIO_PIN_RESET
22         );
23     }
24     else if (num == 2)
25     {
26         /* Number 2 uses segments a, b, d, e, and g */
27         HAL_GPIO_WritePin(
28             GPIOB,
29             GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_3 |
30             GPIO_PIN_4 | GPIO_PIN_6,
31             GPIO_PIN_RESET
32         );
33     }
34 }
35 /* USER CODE END 0 */

```

1.1.5 Initialization in main

After GPIO and TIM2 initialization, both displays are disabled by setting PA6 and PA7 high. The segment pattern for number 1 is then prepared, and PA6 is driven low to enable the first display.

Finally, HAL_TIM_Base_Start_IT(&htim2) starts TIM2 in interrupt mode. The TIM2 interrupt must also be enabled in the CubeMX NVIC settings.

The following excerpt matches the initialization in the supplied source code.

Program 1.2: Exercise 1 initialization inside main

```

1  MX_GPIO_Init();
2  MX_TIM2_Init();
3
4  /* USER CODE BEGIN 2 */
5
6  /* Disable both displays */
7  HAL_GPIO_WritePin(
8      GPIOA,
9      GPIO_PIN_6 | GPIO_PIN_7,
10     GPIO_PIN_SET
11 );

```

```
12
13  /* Prepare number 1, then enable the first display */
14  display7SEG(1);
15  HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_RESET);
16
17  /* Start TIM2 with interrupts */
18  HAL_TIM_Base_Start_IT(&htim2);
19
20  /* USER CODE END 2 */
```

The `while (1)` loop contains no application processing in this exercise. After initialization, display switching is handled by the timer callback, without calling `HAL_Delay()`.

1.1.6 Timer interrupt callback

The callback checks `htim->Instance` so that the display logic executes only for TIM2.

Two static variables retain their values between callback invocations:

- `ticks` counts the 10-ms timer events.
- `active_display` records which display is currently enabled. Value 0 represents the first display, and value 1 represents the second.

The initial value of `active_display` is 0, matching the first display enabled during startup. Every callback increments `ticks`. Once it reaches 50, the counter is reset and the other display is selected.

Program 1.3: Exercise 1 TIM2 callback

```
1  /* USER CODE BEGIN 4 */
2  void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim){
3      if (htim->Instance == TIM2)
4      {
5          static uint8_t ticks = 0;
6          static uint8_t active_display = 0;
7
8          ticks++;
9
10         if (ticks >= 50)
11         {
12             ticks = 0;
13
14             /* Turn off both displays before changing segments */
15             HAL_GPIO_WritePin(
16                 GPIOA,
17                 GPIO_PIN_6 | GPIO_PIN_7,
18                 GPIO_PIN_SET
19             );
20
21             if (active_display == 0)
22             {
23                 /* Change from the first display to the second */
24                 display7SEG(2);
25                 HAL_GPIO_WritePin(
26                     GPIOA, GPIO_PIN_7, GPIO_PIN_RESET
```

```

27         );
28         active_display = 1;
29     }
30     else
31     {
32         /* Change from the second display to the first */
33         display7SEG(1);
34         HAL_GPIO_WritePin(
35             GPIOA, GPIO_PIN_6, GPIO_PIN_RESET
36         );
37         active_display = 0;
38     }
39 }
40 }
41 }
42 /* USER CODE END 4 */

```

Each switching operation follows three steps: disable both displays, write the new segment pattern, and enable the selected display. Disabling the displays before changing the shared segment signals prevents the previous display from briefly showing the new pattern.

The callback executes once for each timer event and then returns. It is not an infinite loop. Between interrupts, the GPIO outputs retain their levels, so the selected display remains enabled.

1.1.7 Expected simulation behavior

Immediately after initialization, the first display shows 1 and the second is disabled. After 50 timer events, approximately 500 ms later, the second display shows 2 and the first is disabled. The sequence then repeats.

Table 2: Nominal display sequence for Exercise 1

Time interval	PA6 / EN0	PA7 / EN1	Visible digit
0–0.5 s	LOW	HIGH	First display: 1
0.5–1.0 s	HIGH	LOW	Second display: 2
1.0–1.5 s	LOW	HIGH	First display: 1
1.5–2.0 s	HIGH	LOW	Second display: 2

Because the required selection interval is 500 ms, the alternating displays are clearly visible in Proteus. This slow scan demonstrates the multiplexing process; it does not make both digits appear continuously illuminated.

1.1.8 Short question: Frequency of the scanning process

The introduction defines the scanning frequency using the time required to visit every display. For N displays, each selected for an interval T_S ,

$$T_{\text{scan}} = NT_S, \quad f_{\text{scan}} = \frac{1}{NT_S}.$$

Exercise 1 has two displays, with $T_S = 500 \text{ ms} = 0.5 \text{ s}$. Therefore,

$$T_{\text{scan}} = 2 \times 0.5 \text{ s} = 1 \text{ s},$$

and

$$f_{\text{scan}} = \frac{1}{1 \text{ s}} = 1 \text{ Hz}.$$

A different display is selected every 0.5 s, giving a selection rate of

$$f_{\text{selection}} = \frac{1}{0.5 \text{ s}} = 2 \text{ Hz.}$$

Thus, the TIM2 interrupt frequency is 100 Hz, the display-selection rate is 2 Hz, and the complete scanning frequency is **1 Hz**.

1.2 Exercise 2: Four digits and the center dot

1.2.1 Objective and GPIO configuration

Exercise 2 extends Exercise 1 by adding two more common-anode seven-segment displays and two center LEDs controlled by the DOT signal on PA4.

The four displays show the digits 1, 2, 3, and 0, representing 12:30. Only one display is enabled at a time, and the selected display changes every 500 ms. Both DOT LEDs change state together every second.

The segment outputs remain connected to PB0–PB6. PA6–PA9 control the four PNP digit-enable circuits.

Table 3: GPIO configuration for Exercise 2

GPIO pins	Signals	Function
PB0–PB6	SEG_A–SEG_G	Shared segment outputs
PA6	EN0	First display: digit 1
PA7	EN1	Second display: digit 2
PA8	EN2	Third display: digit 3
PA9	EN3	Fourth display: digit 0
PA4	DOT	Two center LEDs

The segment outputs, digit enables, and DOT signal are active low. A selected digit is enabled by driving its enable pin low, while the other enable pins remain high.

1.2.2 Schematic

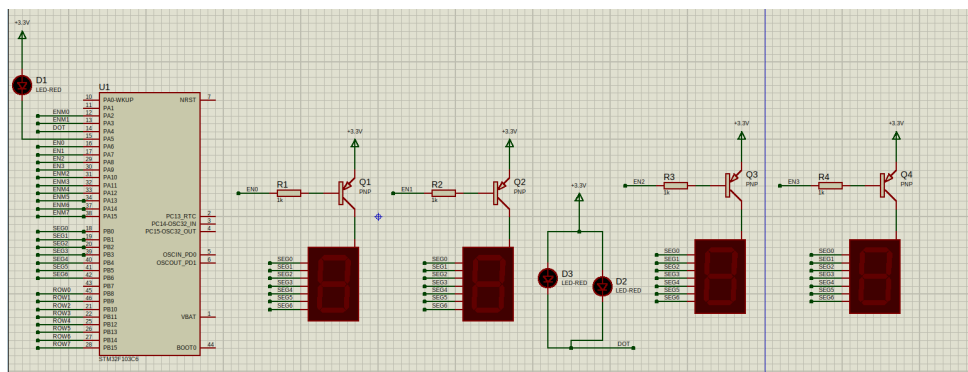


Figure 2: Proteus schematic for Exercise 2: four digits, DOT, and D1.

1.2.3 Step-by-step timing calculation

TIM2 retains the introductory configuration: an 8-MHz input clock, prescaler 7999, and auto-reload value 9.

1.2.3.1 Step 1: Calculate the timer interrupt interval. The counter frequency after the prescaler is

$$f_{\text{CNT}} = \frac{8\,000\,000}{7999 + 1} = 1000 \text{ Hz.}$$

Thus, each counter tick lasts

$$T_{\text{CNT}} = \frac{1}{1000} \text{ s} = 1 \text{ ms.}$$

Since the counter counts from 0 through 9, each update requires ten counter ticks:

$$T_{\text{interrupt}} = (9 + 1) \times 1 \text{ ms} = 10 \text{ ms.}$$

The corresponding interrupt frequency is

$$f_{\text{interrupt}} = \frac{1}{0.01 \text{ s}} = 100 \text{ Hz.}$$

1.2.3.2 Step 2: Calculate the digit-selection threshold. Each display must remain selected for 500 ms. Therefore, the number of interrupts required before selecting the next display is

$$N_{\text{scan}} = \frac{500 \text{ ms}}{10 \text{ ms}} = 50.$$

This gives the callback condition `scan_ticks >= 50`.

1.2.3.3 Step 3: Calculate the DOT-toggle threshold. DOT must change state every second:

$$T_{\text{DOT toggle}} = 1000 \text{ ms.}$$

The required number of interrupts is

$$N_{\text{DOT}} = \frac{1000 \text{ ms}}{10 \text{ ms}} = 100.$$

This gives the callback condition `dot_ticks >= 100`.

The two counters advance during every TIM2 callback, but each counter resets only when its own threshold is reached.

1.2.4 Segment-display function

The supplied Exercise 2 source implements `display7SEG()` for digits 0, 1, 2, and 3. These are the four values required to display 12:30.

Before selecting a new pattern, the function sets all segment outputs high to turn them off. It then drives the required segment outputs low.

Table 4: Segment patterns used in Exercise 2

Digit	Segments turned on
0	a, b, c, d, e, f
1	b, c
2	a, b, d, e, g
3	a, b, c, d, g

Program 1.4: Exercise 2 display7SEG function

```
1  /* USER CODE BEGIN PFP */
2  void display7SEG(int num);
3  /* USER CODE END PFP */
4
5  /* USER CODE BEGIN 0 */
6  void display7SEG(int num)
7  {
8      /* First turn all seven segments off */
9      HAL_GPIO_WritePin(
10         GPIOB,
11         GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_2 | GPIO_PIN_3 |
12         GPIO_PIN_4 | GPIO_PIN_5 | GPIO_PIN_6,
13         GPIO_PIN_SET
14     );
15
16     if (num == 0)
17     {
18         /* 0: a, b, c, d, e, f */
19         HAL_GPIO_WritePin(
20             GPIOB,
21             GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_2 |
22             GPIO_PIN_3 | GPIO_PIN_4 | GPIO_PIN_5,
23             GPIO_PIN_RESET
24         );
25     }
26     else if (num == 1)
27     {
28         /* 1: b, c */
29         HAL_GPIO_WritePin(
30             GPIOB,
31             GPIO_PIN_1 | GPIO_PIN_2,
32             GPIO_PIN_RESET
33         );
34     }
35     else if (num == 2)
36     {
37         /* 2: a, b, d, e, g */
38         HAL_GPIO_WritePin(
39             GPIOB,
40             GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_3 |
41             GPIO_PIN_4 | GPIO_PIN_6,
42             GPIO_PIN_RESET
43         );
44     }
45     else if (num == 3)
46     {
47         /* 3: a, b, c, d, g */
48         HAL_GPIO_WritePin(
49             GPIOB,
50             GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_2 |
51             GPIO_PIN_3 | GPIO_PIN_6,
52             GPIO_PIN_RESET
53         );
54     }
55 }
```

```
56  /* USER CODE END 0 */
```

1.2.5 Initialization in main

After GPIO and TIM2 initialization, all four displays are disabled by setting PA6–PA9 high. PA4 is also set high so that the DOT LEDs start in the off state.

The application then prepares digit 1 and enables the first display by setting PA6 low. Finally, TIM2 is started in interrupt mode.

Program 1.5: Exercise 2 initialization inside main

```
1  MX_GPIO_Init();
2  MX_TIM2_Init();
3
4  /* USER CODE BEGIN 2 */
5
6  /* Disable all four displays */
7  HAL_GPIO_WritePin(
8      GPIOA,
9      GPIO_PIN_6 | GPIO_PIN_7 | GPIO_PIN_8 | GPIO_PIN_9,
10     GPIO_PIN_SET
11 );
12
13 /* Start with the DOT LEDs off */
14 HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, GPIO_PIN_SET);
15
16 /* Show 1 on the first display immediately */
17 display7SEG(1);
18 HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_RESET);
19
20 /* Start TIM2 and its interrupt */
21 HAL_TIM_Base_Start_IT(&htim2);
22
23 /* USER CODE END 2 */
```

The main loop contains no application processing. Digit selection and DOT toggling are both handled in the timer callback without using `HAL_Delay()`.

1.2.6 Timer interrupt callback

The callback checks that the interrupt belongs to TIM2. Three static variables retain their values between callback invocations:

- `scan_ticks` counts timer events for the 500-ms display-selection interval.
- `dot_ticks` counts timer events for the one-second DOT-toggle interval.
- `active_display` identifies the selected display using indices 0–3.

The initial value of `active_display` is 0, matching the first display enabled during startup.

Program 1.6: Exercise 2 TIM2 callback

```
1  /* USER CODE BEGIN 4 */
2  void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
3  {
4      if (htim->Instance == TIM2)
5      {
6          static uint8_t scan_ticks = 0;
7          static uint8_t dot_ticks = 0;
8          static uint8_t active_display = 0;
9
10         /* This callback runs once every 10 ms */
11         scan_ticks++;
12         dot_ticks++;
13
14         /* 100 * 10 ms = 1 second: change the DOT LEDs */
15         if (dot_ticks >= 100)
16         {
17             dot_ticks = 0;
18             HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_4);
19         }
20
21         /* 50 * 10 ms = 500 ms: select the next display */
22         if (scan_ticks >= 50)
23         {
24             scan_ticks = 0;
25
26             /* Disable all displays before changing segments */
27             HAL_GPIO_WritePin(
28                 GPIOA,
29                 GPIO_PIN_6 | GPIO_PIN_7 | GPIO_PIN_8 | GPIO_PIN_9,
30                 GPIO_PIN_SET
31             );
32
33             active_display++;
34             if (active_display >= 4)
35             {
36                 active_display = 0;
37             }
38
39             if (active_display == 0)
40             {
41                 display7SEG(1);
42                 HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_RESET);
43             }
44             else if (active_display == 1)
45             {
46                 display7SEG(2);
47                 HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_RESET);
48             }
49             else if (active_display == 2)
50             {
51                 display7SEG(3);
52                 HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_RESET);
53             }
54             else
55             {
```

```

56         display7SEG(0);
57         HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_RESET);
58     }
59 }
60 }
61 }
62 /* USER CODE END 4 */

```

After every 50 interrupts, the callback disables all displays, advances `active_display`, writes the required digit pattern, and enables the corresponding display. When the index reaches 4, it returns to 0. The repeating selection sequence is therefore

$$0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0.$$

Disabling every display before changing the segment signals prevents the previous display from briefly showing the next digit's pattern.

The DOT and scanning conditions are separate `if` statements. Every 100 interrupts, both conditions are satisfied in the same callback invocation: DOT changes state, and the next display is selected. In the supplied source, DOT is processed first.

1.2.7 Expected simulation behavior

At startup, the first display shows 1 and DOT is off. Every 500 ms, the next display is selected. The digits remain fixed at 1, 2, 3, and 0; this exercise does not advance the displayed time.

Table 5: Nominal display and DOT sequence for Exercise 2

Time interval	Enabled display	Digit	DOT state
0–0.5 s	First, PA6 low	1	Off
0.5–1.0 s	Second, PA7 low	2	Off
1.0–1.5 s	Third, PA8 low	3	On
1.5–2.0 s	Fourth, PA9 low	0	On
2.0–2.5 s	First, PA6 low	1	Off

All unselected digit-enable outputs remain high. The slow 500-ms selection interval makes the scanning sequence visible in Proteus.

DOT stays off for one second and on for one second. Therefore, although its state changes every second, its complete on–off cycle lasts

$$T_{\text{DOT cycle}} = 2 \text{ s}, \quad f_{\text{DOT blink}} = 0.5 \text{ Hz}.$$

1.2.8 Short question: Frequency of the scanning process

Following the definition in the introduction, one complete scan must visit all four displays. Each display is selected for

$$T_S = 500 \text{ ms} = 0.5 \text{ s}.$$

Therefore, the complete scanning period is

$$T_{\text{scan}} = 4T_S = 4 \times 0.5 \text{ s} = 2 \text{ s}.$$

The scanning frequency is

$$f_{\text{scan}} = \frac{1}{T_{\text{scan}}} = \frac{1}{2 \text{ s}} = 0.5 \text{ Hz}.$$

The display-selection rate is

$$f_{\text{selection}} = \frac{1}{T_S} = \frac{1}{0.5 \text{ s}} = 2 \text{ Hz}.$$

Thus, TIM2 interrupts occur at 100 Hz, a new display is selected twice per second, and the complete four-display scanning frequency is **0.5 Hz**.

1.3 Exercise 3: Buffered digit selection

1.3.1 Objective and circuit

Exercise 3 introduces an array named `led_buffer` and a function named `update7SEG(int index)`. The array stores the digits to display, while the function selects a physical display and writes its corresponding array value.

The circuit remains the same as in Figure 2. PB0–PB6 carry the shared segment signals, PA6–PA9 select the four displays, and PA4 controls the DOT LEDs. These signals remain active low.

The initial buffer contains 1, 2, 3, and 4. Display selection remains in the TIM2 callback, with the next display selected every 500 ms. DOT continues to change state every second.

1.3.2 Buffer declarations and function prototypes

The source declares the following variables in `USER CODE BEGIN PV`:

- `MAX_LED` specifies the number of displays.
- `index_led` identifies the currently selected display, using indices 0–3.
- `led_buffer` stores the four digit values.

Program 1.7: Exercise 3 buffer and function prototypes

```
1  /* USER CODE BEGIN PV */
2  const int MAX_LED = 4;
3  int index_led = 0;
4  int led_buffer[4] = {1, 2, 3, 4};
5  /* USER CODE END PV */
6
7  /* USER CODE BEGIN PFP */
8  void display7SEG(int num);
9  void update7SEG(int index);
10 /* USER CODE END PFP */
```

The display index and the digit value have different roles. For example, `update7SEG(2)` selects the third physical display and reads `led_buffer[2]`, whose initial value is 3.

Table 6: Initial buffer-to-display mapping for Exercise 3

Index	Physical display	Enable pin	Buffer value
0	First	PA6	1
1	Second	PA7	2
2	Third	PA8	3
3	Fourth	PA9	4

1.3.3 Digit encoding for values 0–9

The Exercise 2 digit function supports only values 0–3. In Exercise 3, `display7SEG()` is expanded to support all decimal digits, including the value 4 required by the initial buffer.

The function builds a pin mask named `segments_on`. It first sets all segment outputs high, then drives the selected outputs low. An unsupported value leaves all segments off.

Program 1.8: Exercise 3 `display7SEG` function

```
1  /* Put a number from 0 to 9 on the shared PB0-PB6 segment lines */
2  void display7SEG(int num){
3      const uint16_t all_segments =
4          GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_2 | GPIO_PIN_3 |
5          GPIO_PIN_4 | GPIO_PIN_5 | GPIO_PIN_6;
6
7      uint16_t segments_on = 0;
8
9      switch (num)
10     {
11         case 0: /* a b c d e f */
12             segments_on = GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_2 |
13                           GPIO_PIN_3 | GPIO_PIN_4 | GPIO_PIN_5;
14             break;
15
16         case 1: /* b c */
17             segments_on = GPIO_PIN_1 | GPIO_PIN_2;
18             break;
19
20         case 2: /* a b d e g */
21             segments_on = GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_3 |
22                           GPIO_PIN_4 | GPIO_PIN_6;
23             break;
24
25         case 3: /* a b c d g */
26             segments_on = GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_2 |
27                           GPIO_PIN_3 | GPIO_PIN_6;
28             break;
29
30         case 4: /* b c f g */
31             segments_on = GPIO_PIN_1 | GPIO_PIN_2 |
32                           GPIO_PIN_5 | GPIO_PIN_6;
33             break;
34
35         case 5: /* a c d f g */
36             segments_on = GPIO_PIN_0 | GPIO_PIN_2 | GPIO_PIN_3 |
37                           GPIO_PIN_5 | GPIO_PIN_6;
38             break;
39
40         case 6: /* a c d e f g */
41             segments_on = GPIO_PIN_0 | GPIO_PIN_2 | GPIO_PIN_3 |
42                           GPIO_PIN_4 | GPIO_PIN_5 | GPIO_PIN_6;
43             break;
44
45         case 7: /* a b c */
46             segments_on = GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_2;
47             break;
48     }
```

```

49     case 8: /* a b c d e f g */
50         segments_on = all_segments;
51         break;
52
53     case 9: /* a b c d f g */
54         segments_on = GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_2 |
55                     GPIO_PIN_3 | GPIO_PIN_5 | GPIO_PIN_6;
56         break;
57
58     default:
59         break; /* Unsupported number: leave all segments off */
60 }
61
62 /* Common anode: HIGH = segment off, LOW = segment on */
63 HAL_GPIO_WritePin(GPIOB, all_segments, GPIO_PIN_SET);
64
65 if (segments_on != 0)
66 {
67     HAL_GPIO_WritePin(GPIOB, segments_on, GPIO_PIN_RESET);
68 }
69 }

```

1.3.4 Report 1: update7SEG function

The function first checks that the index is between 0 and 3. An invalid index causes it to return without changing the GPIO outputs.

For a valid index, it disables all four displays before updating the shared segment signals. The `switch` statement then reads the corresponding buffer value and enables the selected display.

Both `display7SEG()` and `update7SEG()` are defined inside `USER CODE BEGIN 0`.

Program 1.9: Exercise 3 update7SEG function

```

1  /* Select display index 0, 1, 2, or 3 */
2  void update7SEG(int index){
3      if (index < 0 || index >= MAX_LED)
4      {
5          return;
6      }
7
8      /* Disable every display before changing the shared segments */
9      HAL_GPIO_WritePin(
10         GPIOA,
11         GPIO_PIN_6 | GPIO_PIN_7 | GPIO_PIN_8 | GPIO_PIN_9,
12         GPIO_PIN_SET
13     );
14
15     switch (index)
16     {
17         case 0:
18             display7SEG(led_buffer[0]);
19             HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6, GPIO_PIN_RESET);
20             break;
21
22         case 1:

```

```

23         display7SEG(led_buffer[1]);
24         HAL_GPIO_WritePin(GPIOA, GPIO_PIN_7, GPIO_PIN_RESET);
25         break;
26
27     case 2:
28         display7SEG(led_buffer[2]);
29         HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_RESET);
30         break;
31
32     case 3:
33         display7SEG(led_buffer[3]);
34         HAL_GPIO_WritePin(GPIOA, GPIO_PIN_9, GPIO_PIN_RESET);
35         break;
36
37     default:
38         break;
39 }
40 }

```

Disabling all displays before writing the segment pattern prevents the previously selected display from briefly showing the next digit.

The function selects and updates one display. It does not advance `index_led` or introduce a delay; the callback controls the selection order and timing.

1.3.5 Initialization in main

After GPIO and TIM2 initialization, the application sets DOT high so that its LEDs start off. It then calls `update7SEG(index_led)`. Since `index_led` initially equals 0, the first display immediately shows `led_buffer[0]`, which is 1.

TIM2 is then started in interrupt mode.

Program 1.10: Exercise 3 initialization inside main

```

1  MX_GPIO_Init();
2  MX_TIM2_Init();
3
4  /* USER CODE BEGIN 2 */
5  HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, GPIO_PIN_SET); /* DOT off */
6  update7SEG(index_led); /* Show 1 on display 0 */
7  HAL_TIM_Base_Start_IT(&htim2); /* Start 10 ms interrupts */
8  /* USER CODE END 2 */

```

The main loop contains no application processing in this exercise.

1.3.6 Report 2: Timer interrupt callback

The callback retains two static counters. `scan_ticks` controls display selection, while `dot_ticks` controls DOT toggling. Both counters advance during each 10-ms TIM2 event.

When `scan_ticks` reaches 50, the callback increments `index_led`. If the index reaches `MAX_LED`, it returns to 0. The callback then invokes `update7SEG(index_led)`.

When `dot_ticks` reaches 100, the callback toggles PA4. The DOT and scanning conditions are separate, allowing both tasks to execute during the same callback invocation when required.

Program 1.11: Exercise 3 TIM2 callback

```

1  /* USER CODE BEGIN 4 */
2  void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
3  {
4      if (htim->Instance == TIM2)
5      {
6          static uint8_t scan_ticks = 0;
7          static uint8_t dot_ticks = 0;
8
9          scan_ticks++;
10         dot_ticks++;
11
12         /* Change the DOT LEDs every 100 * 10 ms = 1 second */
13         if (dot_ticks >= 100)
14         {
15             dot_ticks = 0;
16             HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_4);
17         }
18
19         /* Select the next display every 50 * 10 ms = 500 ms */
20         if (scan_ticks >= 50)
21         {
22             scan_ticks = 0;
23
24             index_led++;
25             if (index_led >= MAX_LED)
26             {
27                 index_led = 0;
28             }
29
30             update7SEG(index_led);
31         }
32     }
33 }
34 /* USER CODE END 4 */

```

Unlike Exercise 2, the callback no longer contains the individual digit patterns and enable-pin selections. Those operations are handled by `update7SEG()`, while the displayed values are stored in `led_buffer`.

1.3.7 Expected behavior and buffer testing

With the initial buffer {1, 2, 3, 4}, the displays are selected in physical order: first, second, third, and fourth. Each remains selected for 500 ms before the next display is enabled. DOT changes state every second.

To test the buffer-based selection, change the initializer of `led_buffer`, rebuild the program, and run the updated firmware in Proteus. The following cases provide expected results.

Table 7: Suggested buffer tests for Exercise 3

Buffer initializer	Expected digits, first to fourth
{1, 2, 3, 4}	1, 2, 3, 4
{4, 3, 2, 1}	4, 3, 2, 1
{9, 0, 5, 8}	9, 0, 5, 8

Changing the buffer changes the digit assigned to each display. The physical selection order, selection interval, and DOT timing remain the same.

1.4 Exercise 4: One complete scan per second

1.4.1 Objective and circuit

Exercise 4 changes the display-selection interval so that one complete scan of all four seven-segment displays occurs every second. The required complete scanning frequency is therefore 1 Hz. DOT continues to change state every second.

The circuit remains the same as in Figure 2. The GPIO assignments and the implementations of `display7SEG()` and `update7SEG()` are unchanged from Exercise 3.

The source retains the following buffer declarations.

Program 1.12: Exercise 4 display buffer

```
1  /* USER CODE BEGIN PV */
2  const int MAX_LED = 4;
3  int index_led = 0;
4  int led_buffer[4] = {1, 2, 3, 4};
5  /* USER CODE END PV */
```

The displayed values remain 1, 2, 3, and 4. The change from Exercise 3 is the scanning interval: each display is now selected for 250 ms instead of 500 ms.

1.4.2 Required scan timing

TIM2 retains its 10-ms interrupt interval. The following calculation determines how often the callback must select the next display.

1.4.2.1 Step 1: Determine the complete scanning period. The required scanning frequency is

$$f_{\text{scan}} = 1 \text{ Hz.}$$

Therefore,

$$T_{\text{scan}} = \frac{1}{f_{\text{scan}}} = 1 \text{ s} = 1000 \text{ ms.}$$

1.4.2.2 Step 2: Determine the interval for one display. One complete scan contains four display-selection intervals. Each display must therefore remain selected for

$$T_S = \frac{T_{\text{scan}}}{4} = \frac{1000 \text{ ms}}{4} = 250 \text{ ms.}$$

1.4.2.3 Step 3: Determine the callback counter threshold. Since each TIM2 event occurs every 10 ms, the required number of interrupts is

$$N_{\text{scan}} = \frac{250 \text{ ms}}{10 \text{ ms}} = 25.$$

Thus, the scanning condition changes from `scan_ticks >= 50` in Exercise 3 to `scan_ticks >= 25` in Exercise 4.

1.4.2.4 Step 4: Verify the complete scanning frequency. The four display intervals give

$$T_{\text{scan}} = 4 \times 250 \text{ ms} = 1000 \text{ ms} = 1 \text{ s}.$$

Therefore,

$$f_{\text{scan}} = \frac{1}{1 \text{ s}} = 1 \text{ Hz}.$$

The callback selects a new display four times per second. Each individual display is visited once per second.

1.4.3 Initialization in main

Initialization remains the same as in Exercise 3. DOT starts off, the first display immediately shows `led_buffer[0]`, and TIM2 is started in interrupt mode.

Program 1.13: Exercise 4 initialization inside main

```
1  MX_GPIO_Init();
2  MX_TIM2_Init();
3
4  /* USER CODE BEGIN 2 */
5  HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4, GPIO_PIN_SET); /* DOT off */
6  update7SEG(index_led);                               /* Show 1 on display 0 */
7  HAL_TIM_Base_Start_IT(&htim2);                       /* Start 10 ms interrupts */
8  /* USER CODE END 2 */
```

The main loop contains no application processing. Display selection and DOT toggling remain in the timer callback.

1.4.4 Report 1: Timer interrupt callback

The callback uses two static counters: `scan_ticks` for display selection and `dot_ticks` for DOT toggling. Both counters advance on each TIM2 event.

The DOT threshold remains 100 interrupts. The scanning threshold is reduced to 25 interrupts.

Program 1.14: Exercise 4 TIM2 callback

```
1  /* USER CODE BEGIN 4 */
2  void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
3  {
4      if (htim->Instance == TIM2)
5      {
6          static uint8_t scan_ticks = 0;
7          static uint8_t dot_ticks = 0;
8
9          scan_ticks++;
10         dot_ticks++;
11
12         /* Change the DOT LEDs every 100 * 10 ms = 1 second */
13         if (dot_ticks >= 100)
14         {
15             dot_ticks = 0;
16             HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_4);
```

```

17     }
18
19     /* Select the next display every 25 * 10 ms = 250 ms */
20     if (scan_ticks >= 25)
21     {
22         scan_ticks = 0;
23
24         index_led++;
25         if (index_led >= MAX_LED)
26         {
27             index_led = 0;
28         }
29
30         update7SEG(index_led);
31     }
32 }
33 }
34 /* USER CODE END 4 */

```

When `scan_ticks` reaches 25, it resets to zero. The callback advances `index_led`, returns it to zero when it reaches 4, and calls `update7SEG(index_led)`.

The selected indices repeat in the order

$$0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0.$$

The `update7SEG()` function disables all four displays before changing the shared segment pattern, then enables the selected display.

The DOT and scanning conditions are separate `if` statements. Every 100 interrupts, both conditions are satisfied in the same callback invocation. The source toggles DOT first and then selects the next display.

1.4.5 Expected simulation behavior

The first display shows 1 immediately after initialization. Every 250 ms, the next display is selected. After one second, the selection returns to the first display.

Table 8: Nominal display sequence for Exercise 4

Time interval	Enabled display	Digit	DOT state
0–0.25 s	First, PA6 low	1	Off
0.25–0.50 s	Second, PA7 low	2	Off
0.50–0.75 s	Third, PA8 low	3	Off
0.75–1.00 s	Fourth, PA9 low	4	Off
1.00–1.25 s	First, PA6 low	1	On

Only the selected display is enabled; the other digit-enable outputs remain high. DOT remains off for one second and on for one second, giving a complete blinking cycle of two seconds.

Compared with Exercise 3, the complete scanning frequency increases from 0.5 Hz to 1 Hz. The TIM2 interrupt interval and DOT timing remain unchanged.

The required 1-Hz scan is intentionally slow, so the alternating displays remain visible in Proteus.

1.5 Exercise 5: Digital clock HH:MM

1.5.1 Objective and inherited configuration

Exercise 5 displays a 24-hour digital clock. The first two displays show the hour, and the last two show the minute. Seconds are maintained in software but are not displayed.

The circuit remains as shown in Figure 2. GPIO configuration, TIM2 configuration, `display7SEG()`, `update7SEG()`, and the timer callback are already established in Exercise 4 and are retained.

The callback continues selecting a display every 250 ms and toggling DOT every second. The new clock processing is performed in `main()`.

1.5.2 Time variables and display buffer

The previous buffer declaration is replaced with a volatile buffer initialized to zero, and three time variables are added. The declarations of `MAX_LED` and `index_led` remain as in Exercise 4.

Program 1.15: Exercise 5 updated buffer and new time variables

```
1 volatile int led_buffer[4] = {0, 0, 0, 0};
2
3 int hour = 15;
4 int minute = 8;
5 int second = 50;
```

These declarations belong in `USER CODE BEGIN PV`. The initial time is 15:08:50.

The buffer is written by the main loop and read during interrupt-driven display selection. Its declaration uses `volatile` as in the supplied source.

The new function prototype `void updateClockBuffer(void);` is added to `USER CODE BEGIN PFP`, alongside the existing prototypes.

1.5.3 Report 1: updateClockBuffer function

The function converts the hour and minute into four decimal digits. Integer division by 10 extracts the tens digit, while the remainder after division by 10 extracts the units digit.

Program 1.16: Exercise 5 updateClockBuffer function

```
1 void updateClockBuffer(void){
2     led_buffer[0] = hour / 10;    /* Hour tens */
3     led_buffer[1] = hour % 10;    /* Hour ones */
4     led_buffer[2] = minute / 10;  /* Minute tens */
5     led_buffer[3] = minute % 10;  /* Minute ones */
6 }
```

The definition is placed in `USER CODE BEGIN O`.

For the initial time 15:08, the function produces {1, 5, 0, 8}. A single-digit hour or minute automatically receives a leading zero. For example, 07:05 produces {0, 7, 0, 5}.

1.5.4 Initialization

Initialization follows Exercise 4, with one added call to `updateClockBuffer()` before `update7SEG(index_led)`.

This fills the initially zeroed buffer with {1, 5, 0, 8} before the first display is enabled and TIM2 is started. DOT still starts in the off state.

1.5.5 Clock update in the main loop

The previously empty main loop now waits approximately one second, increments the seconds, handles time rollover, and updates the buffer.

The supplied source places `HAL_Delay(1000)` before `second++`, so the initial time is retained during the first waiting interval.

Program 1.17: Exercise 5 main loop

```
1  /* USER CODE BEGIN WHILE */
2  while (1) {
3      HAL_Delay(1000); /* Wait approximately 1 second */
4      second++;
5
6      if (second >= 60) {
7          second = 0;
8          minute++;
9      }
10
11     if (minute >= 60) {
12         minute = 0;
13         hour++;
14     }
15
16     if (hour >= 24) {
17         hour = 0;
18     }
19
20     updateClockBuffer();
21     /* USER CODE END WHILE */
22
23     /* USER CODE BEGIN 3 */
24 }
25 /* USER CODE END 3 */
```

When seconds reach 60, they reset to zero and the minute increases. When minutes reach 60, they reset to zero and the hour increases. When hours reach 24, they reset to zero.

The separate conditions allow all required rollovers to occur during the same iteration. For example, 23:59:59 advances to 00:00:00.

Although `HAL_Delay()` blocks progress in the main loop, TIM2 interrupts remain enabled. The callback therefore continues scanning the displays and toggling DOT during the delay.

1.5.6 Expected behavior

The clock initially displays 15:08, with 50 seconds stored internally. After approximately ten seconds, the internal time reaches 15:09:00 and the displayed digits change to 15:09.

The hour and minute digits change when their corresponding values advance. Display scanning and DOT blinking continue throughout the clock update process.

1.6 Exercise 6: Software timer for the red LED

1.6.1 Objective and inherited configuration

Exercise 6 introduces a software timer to control the red LED on PA5 and removes the blocking `HAL_Delay()` from the clock-processing loop.

The circuit remains as shown in Figure 2. GPIO and TIM2 configuration, the display buffer, and the functions `display7SEG()`, `update7SEG()`, and `updateClockBuffer()` are retained from Exercise 5.

TIM2 continues generating interrupts every 10 ms. The callback advances the software timer, while the main loop handles the LED and clock events.

1.6.2 New variables and software-timer functions

The following declarations are added to USER CODE BEGIN PV.

Program 1.18: Exercise 6 new variables

```
1 volatile int timer0_counter = 0;
2 volatile int timer0_flag = 0;
3 const int TIMER_CYCLE = 10;      /* TIM2 period: 10 ms */
4 volatile int clock_second_flag = 0;
```

`timer0_counter` stores the remaining number of 10-ms timer events. `timer0_flag` indicates expiry. The separate `clock_second_flag` notifies the main loop that one second has passed.

The following functions are added to USER CODE BEGIN 0.

Program 1.19: Exercise 6 software-timer functions

```
1 void setTimer0(int duration)
2 {
3     timer0_counter = duration / TIMER_CYCLE;
4     timer0_flag = 0;
5 }
6
7 void timer_run(void)
8 {
9     if (timer0_counter > 0)
10    {
11        timer0_counter--;
12
13        if (timer0_counter == 0)
14        {
15            timer0_flag = 1;
16        }
17    }
18 }
```

`setTimer0()` converts a duration in milliseconds into timer events and clears the previous expiry flag.

`timer_run()` decreases a positive counter once per callback. When it reaches zero, the function raises the flag. The main loop performs the LED action.

1.6.3 Updated timer interrupt callback

The function `HAL_TIM_PeriodElapsedCallback()` is modified to call `timer_run()` and raise `clock_second_flag`.

Display selection and DOT toggling remain inside the callback.

Program 1.20: Exercise 6 updated timer callback

```
1  /* USER CODE BEGIN 4 */
2  void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
3  {
4      if (htim->Instance == TIM2)
5      {
6          static uint8_t scan_ticks = 0;
7          static uint8_t dot_ticks = 0;
8
9          timer_run();    /* Decrease software timer once per 10 ms */
10
11         scan_ticks++;
12         dot_ticks++;
13
14         if (dot_ticks >= 100)
15         {
16             dot_ticks = 0;
17             HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_4);
18             clock_second_flag = 1; /* Tell main one second passed */
19         }
20
21         if (scan_ticks >= 25)
22         {
23             scan_ticks = 0;
24
25             index_led++;
26             if (index_led >= MAX_LED)
27             {
28                 index_led = 0;
29             }
30
31             update7SEG(index_led);
32         }
33     }
34 }
35 /* USER CODE END 4 */
```

The timer-instance check restricts processing to TIM2. Each callback advances timer 0. Every 100 callbacks, DOT changes state and the clock flag is raised. Every 25 callbacks, the next display is selected.

Thus, the clock notification is generated by TIM2, while the main loop performs the time update.

1.6.4 Initialization

The display initialization follows Exercise 5. PA5 is also set high so that the red LED starts off, and timer 0 is initialized before TIM2 starts.

Program 1.21: Exercise 6 initialization

```
1  /* USER CODE BEGIN 2 */
2  HAL_GPIO_WritePin(
3      GPIOA, GPIO_PIN_4 | GPIO_PIN_5, GPIO_PIN_SET
4  );
5
6  updateClockBuffer();
7  update7SEG(index_led);
8
9  setTimer0(1000);
10 HAL_TIM_Base_Start_IT(&htim2);
11 /* USER CODE END 2 */
```

1.6.5 Updated main loop

The main loop checks two independent flags. Timer 0 controls the PA5 LED, while `clock_second_flag` controls the clock update.

Program 1.22: Exercise 6 main loop

```
1  /* USER CODE BEGIN WHILE */
2  while (1) {
3      if (timer0_flag == 1) {
4          HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
5          setTimer0(2000);
6      }
7
8      if (clock_second_flag == 1) {
9          clock_second_flag = 0;
10
11          second++;
12
13          if (second >= 60) {
14              second = 0;
15              minute++;
16          }
17
18          if (minute >= 60) {
19              minute = 0;
20              hour++;
21          }
22
23          if (hour >= 24) {
24              hour = 0;
25          }
26
27          updateClockBuffer();
28      }
29      /* USER CODE END WHILE */
30
31      /* USER CODE BEGIN 3 */
32  }
33  /* USER CODE END 3 */
```

The first LED toggle occurs after approximately one second. Subsequent toggles occur every two seconds because the main loop rearms timer 0 with `setTimer0(2000)`. This call also clears its expiry flag.

The clock-rollover logic follows Exercise 5. It now executes when the clock flag is raised, without a blocking delay.

1.6.6 Answers to the three report questions

Each question changes only the initial `setTimer0(1000)` call. The later `setTimer0(2000)` call remains unchanged.

Table 9: Effect of changing the initial software-timer call

Initial call	Outcome and reason
Omitted	The counter and flag remain zero. Since <code>timer_run()</code> only processes positive counters, no expiry flag is raised. The PA5 LED remains off.
<code>setTimer0(1)</code>	Integer division gives $1 / 10 = 0$, and the flag is cleared. The zero counter is not decremented, so the flag is never raised. The PA5 LED remains off.
<code>setTimer0(10)</code>	Integer division gives $10 / 10 = 1$. The next TIM2 callback decrements the counter to zero and raises the flag. The first toggle occurs after approximately 10 ms. Subsequent toggles occur every two seconds after rearming.

These changes affect the PA5 task. Display scanning, DOT blinking, and clock updates continue independently of timer 0.

1.6.7 Expected behavior

PA5 initially remains off, turns on after approximately one second, and then changes state every two seconds.

The initial clock time is 15:08:50. The displayed time becomes 15:09 after approximately ten seconds. Display selection continues every 250 ms, and DOT changes state every second.

1.7 Exercise 7: Clock and DOT handled in main

1.7.1 Objective and inherited configuration

Exercise 7 uses a software timer for the clock and moves DOT toggling to the main loop. The circuit remains as shown in Figure 2.

GPIO and TIM2 configuration, display functions, clock-buffer conversion, and `setTimer0()` are already established in the previous exercises and are retained.

In the supplied source, timer 0 is reassigned to the one-second clock and DOT task. The previous `clock_second_flag` is removed.

1.7.2 Software-timer extension

The source adds the following declarations to `USER CODE BEGIN PV`.

Program 1.23: Exercise 7 additional timer variables

```
1 volatile int timer1_counter = 0;
```

```
2 volatile int timer1_flag = 0;
```

The existing timer 0 declarations and `TIMER_CYCLE` remain unchanged.

A second timer-setting function is added, and `timer_run()` is extended to process both counters.

Program 1.24: Exercise 7 additional timer and updated `timer_run`

```
1 void setTimer1(int duration)
2 {
3     timer1_counter = duration / TIMER_CYCLE;
4     timer1_flag = 0;
5 }
6
7 void timer_run(void)
8 {
9     if (timer0_counter > 0)
10    {
11        timer0_counter--;
12
13        if (timer0_counter == 0)
14        {
15            timer0_flag = 1;
16        }
17    }
18
19    if (timer1_counter > 0)
20    {
21        timer1_counter--;
22
23        if (timer1_counter == 0)
24        {
25            timer1_flag = 1;
26        }
27    }
28 }
```

These functions are defined in `USER CODE BEGIN 0`. Both timers follow the countdown mechanism established in Exercise 6.

1.7.3 Updated timer interrupt callback

The function `HAL_TIM_PeriodElapsedCallback()` is modified again. The DOT counter and DOT-toggle block are removed.

The callback now advances the software timers and maintains the existing display scan.

Program 1.25: Exercise 7 updated timer callback

```
1 /* USER CODE BEGIN 4 */
2 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
3 {
4     if (htim->Instance == TIM2)
```

```

5      {
6          static uint8_t scan_ticks = 0;
7
8          timer_run();    /* Count down timer0 and timer1 every 10 ms */
9
10         scan_ticks++;
11         if (scan_ticks >= 25)
12         {
13             scan_ticks = 0;
14
15             index_led++;
16             if (index_led >= MAX_LED)
17             {
18                 index_led = 0;
19             }
20
21             update7SEG(index_led);
22         }
23     }
24 }
25 /* USER CODE END 4 */

```

Display selection remains inside the callback and occurs every 250 ms. Clock updates and DOT toggling are performed by the main loop when timer 0 expires.

1.7.4 Initialization

Initialization follows Exercise 6, with both software timers initially set to 1000 ms.

Program 1.26: Exercise 7 initialization

```

1  /* USER CODE BEGIN 2 */
2  HAL_GPIO_WritePin(
3      GPIOA, GPIO_PIN_4 | GPIO_PIN_5, GPIO_PIN_SET
4  );    /* DOT and red LED initially off */
5
6  updateClockBuffer();
7  update7SEG(index_led);    /* Select the first clock digit */
8
9  setTimer0(1000);    /* Clock and DOT timer */
10 setTimer1(1000);    /* Initialized but not handled in main */
11
12 HAL_TIM_Base_Start_IT(&htim2);
13 /* USER CODE END 2 */

```

1.7.5 Report 1: Main-loop source code

The main loop checks `timer0_flag`. When timer 0 expires, it rearms the timer, advances the clock, updates the display buffer, and toggles DOT.

Program 1.27: Exercise 7 main loop

```
1  /* USER CODE BEGIN WHILE */
2  while (1) {
3      /* USER CODE END WHILE */
4
5      /* USER CODE BEGIN 3 */
6      if (timer0_flag == 1) {
7          /* Clear the flag and schedule the next second. */
8          setTimer0(1000);
9
10         second++;
11
12         if (second >= 60) {
13             second = 0;
14             minute++;
15         }
16
17         if (minute >= 60) {
18             minute = 0;
19             hour++;
20         }
21
22         if (hour >= 24) {
23             hour = 0;
24         }
25
26         updateClockBuffer();
27         HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_4);
28     }
29 } /* Closes while (1). */
30 /* USER CODE END 3 */
```

Calling `setTimer0(1000)` clears the expiry flag and schedules the next clock event. The rollover logic is retained from Exercise 5.

DOT now changes state during the same main-loop iteration as the clock update. No `HAL_Delay()` is required.

1.7.6 Expected behavior and timer usage

The clock starts at 15:08:50 and advances approximately once per second. DOT also changes state once per second. The displayed time becomes 15:09 after approximately ten seconds.

In the supplied source, the main loop checks only `timer0_flag`. Timer 1 expires once, and its flag remains set because no application task handles it.

The main loop contains no PA5 toggle operation. The red LED therefore remains off, while timer 0 controls the clock and DOT.

1.8 Exercise 8: Software-timer-only TIM2 callback

1.8.1 Objective and inherited configuration

Exercise 8 moves periodic display selection from the timer callback to the main loop. The interrupt callback now only advances the software timers.

The circuit remains as shown in Figure 2. GPIO and TIM2 configuration match Exercise 7.

The functions `display7SEG()`, `update7SEG()`, and `updateClockBuffer()` match exactly the same functions in Exercise 7. The functions `setTimer0()` and `setTimer1()` also retain the same logic.

The supplied source assigns the three software timers as follows.

Table 10: Software-timer assignments for Exercise 8

Timer	Task	Duration
Timer 0	PA5 red LED	1000 ms initially; 2000 ms thereafter
Timer 1	Clock update and DOT	1000 ms
Timer 2	Display selection	250 ms

Unlike the supplied Exercise 7 source, the clock task now uses timer 1, and the PA5 red-LED task is active again.

1.8.2 Additional software timer

The following declarations are added to `USER CODE BEGIN PV`. All other variable declarations retain their values from Exercise 7.

Program 1.28: Exercise 8 additional timer variables

```
1 volatile int timer2_counter = 0;
2 volatile int timer2_flag = 0;
```

The new function `setTimer2()` is defined in `USER CODE BEGIN 0`. It follows the same mechanism as `setTimer0()` and `setTimer1()` in Exercise 7.

Program 1.29: Exercise 8 setTimer2 function

```
1 void setTimer2(int duration) {
2     timer2_counter = duration / TIMER_CYCLE;
3     timer2_flag = 0;
4 }
```

The timer 0 and timer 1 countdown blocks inside `timer_run()` retain the same logic as Exercise 7. The following block is added after them, before the function's closing brace.

Program 1.30: Additional timer 2 block inside timer_run

```
1 if (timer2_counter > 0) {
2     timer2_counter--;
3     if (timer2_counter == 0) {
4         timer2_flag = 1;
5     }
6 }
```

Thus, each call to `timer_run()` advances all three software timers and raises the corresponding expiry flags.

1.8.3 Updated timer interrupt callback

The function `HAL_TIM_PeriodElapsedCallback()` is changed from Exercise 7. The `scan_ticks` counter and display-selection block are removed.

Program 1.31: Exercise 8 updated timer callback

```
1  /* USER CODE BEGIN 4 */
2  void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
3  {
4      if (htim->Instance == TIM2)
5      {
6          timer_run();
7      }
8  }
9  /* USER CODE END 4 */
```

The callback checks the timer instance and advances the software timers every 10 ms. LED toggling, clock updates, and periodic display selection are handled in the main loop.

1.8.4 Initialization

Initialization matches Exercise 7: DOT and PA5 start off, the clock buffer is prepared, the first display is selected, and timers 0 and 1 are initialized to 1000 ms.

One additional call is inserted after `setTimer1(1000)` and before `HAL_TIM_Base_Start_IT(&htim2)`.

Program 1.32: Exercise 8 additional initialization

```
1  setTimer2(250); /* Next display selection */
```

1.8.5 Report 1: Main-loop source code

The main loop now checks three independent expiry flags. The clock-rollover logic matches Exercise 7, but it is triggered by `timer1_flag`.

Program 1.33: Exercise 8 main loop

```
1  /* USER CODE BEGIN WHILE */
2  while (1) {
3      /* Red LED: first toggle after 1 s, then every 2 s */
4      if (timer0_flag == 1) {
5          HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
6          setTimer0(2000);
7      }
8
9      /* Digital clock and DOT: update every 1 s */
10     if (timer1_flag == 1) {
11         setTimer1(1000);
12
13         second++;
14
15         if (second >= 60) {
```

```
16         second = 0;
17         minute++;
18     }
19
20     if (minute >= 60) {
21         minute = 0;
22         hour++;
23     }
24
25     if (hour >= 24) {
26         hour = 0;
27     }
28
29     updateClockBuffer();
30     HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_4);
31 }
32
33 /* Seven-segment scan: change display every 250 ms */
34 if (timer2_flag == 1) {
35     setTimer2(250);
36
37     index_led++;
38     if (index_led >= MAX_LED) {
39         index_led = 0;
40     }
41
42     update7SEG(index_led);
43 }
44 /* USER CODE END WHILE */
45
46 /* USER CODE BEGIN 3 */
47 }
48 /* USER CODE END 3 */
```

Each timer-setting call clears its expiry flag and schedules the next event.

Timer 2 advances `index_led` through indices 0–3 and calls `update7SEG(index_led)`. The display-selection function itself matches Exercise 7; its periodic invocation has moved to the main loop.

The three conditions are separate `if` statements, allowing multiple expired tasks to be handled during the same loop iteration.

1.8.6 Expected behavior

The clock initially displays 15:08 and advances approximately once per second. DOT changes state with each clock update.

The selected display changes every 250 ms. PA5 starts off, toggles after approximately one second, and then changes state every two seconds.

All tasks run without `HAL_Delay()`, while the TIM2 callback only maintains the software-timer countdowns.

1.9 Exercise 9: Display the letter A on an 8×8 matrix

1.9.1 Objective and inherited configuration

Exercise 9 adds an 8 × 8 LED matrix and a ULN2803 column-control stage to display a stationary capital A.

The functions `display7SEG()`, `update7SEG()`, `updateClockBuffer()`, `setTimer0()`, `setTimer1()`, `setTimer2()`, and `timer_run()` match exactly the same functions in Exercise 8.

TIM2 configuration and `HAL_TIM_PeriodElapsedCallback()` also match Exercise 8. TIM2 continues interrupting every 10 ms, and its callback only calls `timer_run()`.

Matrix scanning is added to the main loop, alongside the existing clock and LED tasks.

1.9.2 Updated matrix GPIO configuration

The updated circuit connects ENM0 to PA2 and ENM1 to PA3. The remaining column assignments are shown in Table 11.

Table 11: Updated GPIO assignments for the LED matrix

Column index	Enable signal	GPIO pin
0	ENM0	PA2
1	ENM1	PA3
2	ENM2	PA10
3	ENM3	PA11
4	ENM4	PA12
5	ENM5	PA13
6	ENM6	PA14
7	ENM7	PA15

All column-control signals use GPIOA. ROW0–ROW7 connect to PB8–PB15, respectively. The updated definitions in `main.h` include:

Program 1.34: Updated ENM0 and ENM1 pin definitions

```
1 #define ENM0_Pin GPIO_PIN_2
2 #define ENM0_GPIO_Port GPIOA
3 #define ENM1_Pin GPIO_PIN_3
4 #define ENM1_GPIO_Port GPIOA
```

The generated `MX_GPIO_Init()` function uses these pin definitions to configure the matrix outputs as push-pull, with no internal pull resistors and an initial HIGH output level.

The matrix-display function also uses the ENM pin definitions. GPIO initialization and column selection therefore follow the same mapping.

In the implemented circuit, HIGH on an ENM output disables its column, while LOW selects it. For the selected column, a LOW row output lights the corresponding pixel and a HIGH row output turns it off.

1.9.3 Report 1: Schematic

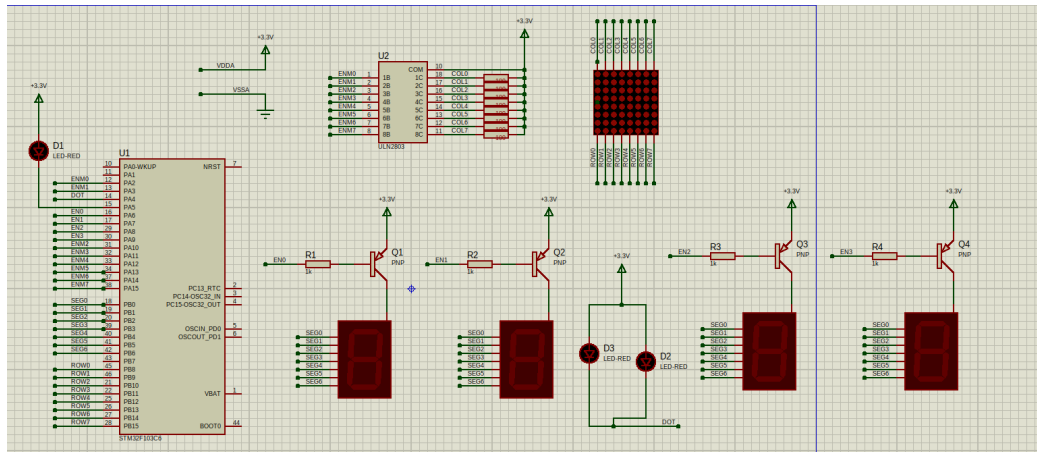


Figure 3: Exercise 9 Proteus circuit with the 8×8 LED matrix and ULN2803.

1.9.4 Matrix variables and character pattern

The following declarations are added inside `USER CODE BEGIN PV`. The existing clock, display-buffer, and software-timer variables match Exercise 8.

Program 1.35: Exercise 9 matrix variables and smaller A

```
1  const int MAX_LED_MATRIX = 8;
2  int index_led_matrix = 0;
3
4  /* Smaller A: 5 columns wide, 6 rows tall, with blank margins */
5  uint8_t matrix_buffer[8] = {
6      0x00, 0x7C, 0x12, 0x12,
7      0x12, 0x7C, 0x00, 0x00
8  };
9
10 uint32_t last_matrix_scan = 0;
```

Each buffer element represents one column. Its eight bits specify the pixels on ROW0–ROW7. Bit 0 corresponds to PB8, and bit 7 corresponds to PB15.

A stored bit value of 1 represents a lit pixel and produces a LOW row output. A stored bit value of 0 produces a HIGH row output.

The A occupies five columns and six rows. The remaining positions provide blank margins.

1.9.5 Report 2: updateLEDMatrix function

The function `updateLEDMatrix()` is defined inside `USER CODE BEGIN 0`. It checks the index, disables all columns, writes the row pattern, and enables the selected column.

Program 1.36: Exercise 9 updateLEDMatrix function

```
1  void updateLEDMatrix(int index) {
2      const uint16_t all_columns = ENM0_Pin | ENM1_Pin | ENM2_Pin | ENM3_Pin |
3                                  ENM4_Pin | ENM5_Pin | ENM6_Pin | ENM7_Pin;
```

```
4
5     uint16_t selected_column;
6
7     if (index < 0 || index >= MAX_LED_MATRIX) {
8         return;
9     }
10
11     /* Disable all columns before changing the row data. */
12     HAL_GPIO_WritePin(GPIOA, all_columns, GPIO_PIN_SET);
13
14     /* Update the eight active-low row outputs. */
15     for (int row = 0; row < 8; row++) {
16         uint16_t row_pin = (uint16_t)(GPIO_PIN_8 << row);
17
18         GPIO_PinState row_state = (matrix_buffer[index] & (1U << row))
19                                     ? GPIO_PIN_RESET /* Pixel ON */
20                                     : GPIO_PIN_SET; /* Pixel OFF */
21
22         HAL_GPIO_WritePin(GPIOB, row_pin, row_state);
23     }
24
25     /* Select the column using the definitions in main.h. */
26     switch (index) {
27     case 0:
28         selected_column = ENM0_Pin; /* PA2 */
29         break;
30
31     case 1:
32         selected_column = ENM1_Pin; /* PA3 */
33         break;
34
35     case 2:
36         selected_column = ENM2_Pin; /* PA10 */
37         break;
38
39     case 3:
40         selected_column = ENM3_Pin; /* PA11 */
41         break;
42
43     case 4:
44         selected_column = ENM4_Pin; /* PA12 */
45         break;
46
47     case 5:
48         selected_column = ENM5_Pin; /* PA13 */
49         break;
50
51     case 6:
52         selected_column = ENM6_Pin; /* PA14 */
53         break;
54
55     case 7:
56         selected_column = ENM7_Pin; /* PA15 */
57         break;
58
59     default:
```

```
60     return;
61 }
62
63 /* Enable only the selected column: active LOW. */
64 HAL_GPIO_WritePin(GPIOA, selected_column, GPIO_PIN_RESET);
65 }
```

Both the all-column mask and the selection cases use ENM0_Pin–ENM7_Pin. Column 0 selects PA2, and column 1 selects PA3.

The row loop uses consecutive GPIO pins PB8–PB15. Disabling every column before changing these outputs prevents the previous column from briefly displaying the next column’s pattern.

1.9.6 Complete application initialization

After the existing GPIO and TIM2 initialization, the application prepares the clock, initializes the three software timers, and starts TIM2. It then selects the initial matrix column and records the matrix-scan timestamp.

Program 1.37: Exercise 9 initialization inside main

```
1  /* USER CODE BEGIN 2 */
2  HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4 | GPIO_PIN_5, GPIO_PIN_SET);
3
4  updateClockBuffer();
5  update7SEG(index_led); /* Show the first digit immediately */
6
7  setTimer0(1000); /* PA5's first toggle */
8  setTimer1(1000); /* Next clock second and DOT toggle */
9  setTimer2(250); /* Next display selection */
10
11 HAL_TIM_Base_Start_IT(&htim2);
12
13 updateLEDMatrix(index_led_matrix);
14 last_matrix_scan = HAL_GetTick();
15 /* USER CODE END 2 */
```

The initial matrix index is zero, so the first selected column is ENM0 on PA2.

1.9.7 Complete main loop

The complete loop below includes all inherited tasks and the new matrix-scanning task. The red-LED, clock, DOT, and seven-segment processing follows Exercise 8.

Program 1.38: Exercise 9 complete main loop

```
1  /* USER CODE BEGIN WHILE */
2  while (1) {
3      /* Red LED: first toggle after 1 s, then every 2 s */
4      if (timer0_flag == 1) {
5          HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
6          setTimer0(2000);
```

```
7     }
8
9     /* Digital clock and DOT: update every 1 s */
10    if (timer1_flag == 1) {
11        setTimer1(1000);
12
13        second++;
14
15        if (second >= 60) {
16            second = 0;
17            minute++;
18        }
19
20        if (minute >= 60) {
21            minute = 0;
22            hour++;
23        }
24
25        if (hour >= 24) {
26            hour = 0;
27        }
28
29        updateClockBuffer();
30        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_4);
31    }
32
33    /* Seven-segment scan: change display every 250 ms */
34    if (timer2_flag == 1) {
35        setTimer2(250);
36
37        index_led++;
38        if (index_led >= MAX_LED) {
39            index_led = 0;
40        }
41
42        update7SEG(index_led);
43    }
44
45    /* Exercise 09: change matrix column every 5 ms */
46    uint32_t now = HAL_GetTick();
47
48    if ((uint32_t)(now - last_matrix_scan) >= 5U) {
49        last_matrix_scan = now;
50
51        index_led_matrix++;
52        if (index_led_matrix >= MAX_LED_MATRIX) {
53            index_led_matrix = 0;
54        }
55
56        updateLEDMatrix(index_led_matrix);
57    }
58    /* USER CODE END WHILE */
59
60    /* USER CODE BEGIN 3 */
61 }
62 /* USER CODE END 3 */
```

Timer 0 controls PA5, timer 1 controls the clock and DOT, and timer 2 controls seven-segment display selection.

Matrix scanning uses the HAL millisecond time base. The comparison with 5U selects a nominal 5-ms column interval. The index advances through columns 0–7 and returns to zero after column 7.

The stored A pattern remains unchanged. The GPIO outputs retain each selected column and its row pattern between updates. No `HAL_Delay()` is used.

1.9.8 Expected behavior

The smaller A remains stationary while its columns are repeatedly scanned.

The clock advances approximately once per second, DOT changes state every second, and the selected seven-segment display changes every 250 ms.

PA5 starts off, toggles after approximately one second, and then changes state every two seconds.

1.10 Exercise 10: Scroll the matrix character to the left

1.10.1 Objective and inherited configuration

Exercise 10 adds left-scrolling movement to the A displayed in Exercise 9.

The circuit remains as shown in Figure 3. GPIO assignments match Table 11: ENM0 uses PA2, ENM1 uses PA3, and ENM2–ENM7 use PA10–PA15.

The `updateLEDMatrix()` function, initial matrix buffer, clock functions, software-timer functions, GPIO initialization, and `HAL_TIM_PeriodElapsedCallback()` match exactly the corresponding parts in Exercise 9.

Column scanning retains its 5-ms interval. The new animation uses a nominal 200-ms movement interval.

1.10.2 Additional animation variables

The matrix declarations from Exercise 9 are retained. The following declarations are added inside `USER CODE BEGIN PV`.

Program 1.39: Exercise 10 animation variables and source pattern

```
1  uint32_t last_matrix_animation = 0;
2  int matrix_scroll_offset = 0;
3
4  /* Small A with margins, followed by eight blank columns */
5  const uint8_t scrolling_pattern[16] = {
6      0x00, 0x7C, 0x12, 0x12,
7      0x12, 0x7C, 0x00, 0x00,
8
9      0x00, 0x00, 0x00, 0x00,
10     0x00, 0x00, 0x00, 0x00
11 };
```

The first eight source bytes match the initial A frame from Exercise 9. The next eight bytes provide blank columns between successive appearances of the character.

The matrix displays an eight-column window from this 16-column source. `matrix_scroll_offset` determines the starting position of that window.

1.10.3 Report 1: Left-scrolling function

The new function `scrollMatrixLeft()` is defined inside `USER CODE BEGIN 0`.

Program 1.40: Exercise 10 `scrollMatrixLeft` function

```
1 void scrollMatrixLeft(void) {
2     matrix_scroll_offset++;
3
4     if (matrix_scroll_offset >= 16) {
5         matrix_scroll_offset = 0;
6     }
7
8     for (int col = 0; col < MAX_LED_MATRIX; col++) {
9         int source = (col + matrix_scroll_offset) % 16;
10        matrix_buffer[col] = scrolling_pattern[source];
11    }
12 }
```

Each call advances the viewing-window offset by one and rebuilds all eight display columns. Each physical column receives the next source column, making the visible character move left.

The modulo operation wraps the viewing window around the 16-column source. After 16 movements, the initial position repeats.

The function changes the buffer contents. The inherited `updateLEDMatrix()` function displays those contents using the updated ENM pin definitions.

1.10.4 Complete application initialization

Initialization follows Exercise 9, with the animation timestamp initialized from the matrix-scan timestamp.

Program 1.41: Exercise 10 initialization inside main

```
1  /* USER CODE BEGIN 2 */
2  HAL_GPIO_WritePin(GPIOA, GPIO_PIN_4 | GPIO_PIN_5, GPIO_PIN_SET);
3
4  updateClockBuffer();
5  update7SEG(index_led); /* Show the first digit immediately */
6
7  setTimer0(1000); /* PA5's first toggle */
8  setTimer1(1000); /* Next clock second and DOT toggle */
9  setTimer2(250); /* Next display selection */
10
11 HAL_TIM_Base_Start_IT(&htim2);
12
13 updateLEDMatrix(index_led_matrix);
14 last_matrix_scan = HAL_GetTick();
15
16 last_matrix_animation = last_matrix_scan;
17 /* USER CODE END 2 */
```

The scroll offset initially equals zero. The initial buffer therefore shows the same A position as Exercise 9.

1.10.5 Complete main loop

The complete loop retains all inherited tasks and adds animation inside the matrix-scanning block.

Program 1.42: Exercise 10 complete main loop

```
1  /* USER CODE BEGIN WHILE */
2  while (1) {
3      /* Red LED: first toggle after 1 s, then every 2 s */
4      if (timer0_flag == 1) {
5          HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
6          setTimer0(2000);
7      }
8
9      /* Digital clock and DOT: update every 1 s */
10     if (timer1_flag == 1) {
11         setTimer1(1000);
12
13         second++;
14
15         if (second >= 60) {
16             second = 0;
17             minute++;
18         }
19
20         if (minute >= 60) {
21             minute = 0;
22             hour++;
23         }
24
25         if (hour >= 24) {
26             hour = 0;
27         }
28
29         updateClockBuffer();
30         HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_4);
31     }
32
33     /* Seven-segment scan: change display every 250 ms */
34     if (timer2_flag == 1) {
35         setTimer2(250);
36
37         index_led++;
38         if (index_led >= MAX_LED) {
39             index_led = 0;
40         }
41
42         update7SEG(index_led);
43     }
44
45     uint32_t now = HAL_GetTick();
46
47     /* Scan one matrix column every 5 ms */
48     if ((uint32_t)(now - last_matrix_scan) >= 5U) {
49         last_matrix_scan = now;
50     }
```

```
51     index_led_matrix++;
52     if (index_led_matrix >= MAX_LED_MATRIX) {
53         index_led_matrix = 0;
54     }
55
56     /* Move A at the beginning of a complete scan */
57     if (index_led_matrix == 0 &&
58         (uint32_t)(now - last_matrix_animation) >= 200U) {
59
60         last_matrix_animation += 200U;
61         scrollMatrixLeft();
62     }
63
64     updateLEDMatrix(index_led_matrix);
65 }
66 /* USER CODE END WHILE */
67
68 /* USER CODE BEGIN 3 */
69 }
70 /* USER CODE END 3 */
```

The red-LED, clock, DOT, and seven-segment blocks retain the behavior of Exercise 9.

The outer matrix condition controls the 5-ms column interval. The inner condition allows movement when the animation deadline has been reached and the next displayed column is column 0.

The buffer is updated immediately before displaying column 0 through ENM0 on PA2. It remains unchanged while the remaining columns of that scan are displayed. This prevents the character position from changing halfway through a complete scan.

The statement `last_matrix_animation += 200U` advances the scheduled movement time by a fixed interval. Actual movement occurs at the first eligible beginning of a scan.

Scanning and animation both remain in the main loop. The TIM2 callback continues calling only `timer_run()`.

1.10.6 Expected behavior

The smaller A begins in the same position as in Exercise 9. It moves left one column at a time, leaves the display, passes through the blank portion of the source, and re-enters from the right.

The character keeps its original shape. The clock, DOT, PA5 red LED, and seven-segment tasks continue operating during the animation, without a blocking delay.

References

- [1] V. L. T. Trieu, *Lab 02 code + schematic*, GitHub repository. [Online]. Available: <https://github.com/volethanhtrieu/Lab-exercise-MPU---MCU/lab02>.
- [2] Ho Chi Minh City University of Technology, *Microcontroller Laboratory – Lab 02: Timer Interrupt and LED Scanning*, course laboratory manual.
- [3] STMicroelectronics, *STM32F103x4 and STM32F103x6 Low-density Performance Line Datasheet*, DocID15060, Rev. 7, June 2015. [Online]. Available: <https://www.st.com/resource/en/datasheet/stm32f103c4.pdf>. [Accessed: September 18, 2026].
- [4] STMicroelectronics, *STM32CubeMX: STM32Cube Initialization Code Generator*. [Online]. Available: <https://www.st.com/en/development-tools/stm32cubemx.html>. [Accessed: September 18, 2026].
- [5] STMicroelectronics, *STM32CubeF1 Embedded Software Package*. [Online]. Available: <https://www.st.com/en/embedded-software/stm32cubef1.html>. [Accessed: September 18, 2026].
- [6] Labcenter Electronics, *Proteus Circuit Simulation Software with SPICE*. [Online]. Available: <https://www.labcenter.com/simulation/>. [Accessed: September 18, 2026].